

בס"ד

RecipeBox

ספר פרויקט גמר
Full-Stack

React · TypeScript · ASP.NET Core · PostgreSQL · Google AI Studio

רבקי גבאי | סמינר וולף | 2026

מנחה: שרה אבער



אהל אברהם

עמותה מס' 58-009-812-7

ע"י סמינר בית יעקב ע"ש הרב י. א. וולף זצ"ל
רחוב הרב הירש 27 בני-ברק מ. 51510 ת.ד. 542 טל. 6183211 פקס. 5797608



תאריך: 18.02.26

לכבוד יחידת הפרויקטים מה"ט

הצעה לפרויקט גמר

א. פרטי הסטודנטים

שם הסטודנט	ת.ז.	כתובת	טלפון נייד	שנת סיום הלימודים
רבקי גבאי	214718702	גינות דוד בני ברק	055-6757343	2026

שם המכללה: סמינר אהל אברהם

סמל המכללה: 76078

מסלול ההכשרה: הנדסאים

מגמת לימוד: הנדסת תוכנה

מקום ביצוע הפרויקט:

ב. פרטי המנחה האישי

שם המנחה *	כתובת	טלפון נייד	תואר	מקום עבודה/תפקיד
מלכה ברוק	הרב מצליח 10 ב"ב	052-7603743		

רבקי גבאי

חתימת הגורם המקצועי
מטעם מה"ט

חתימת המנחה האישי

חתימת הסטודנטים

1. שם הפרויקט:

RecipeBox – מערכת מתקדמת לניהול ושיתוף מתכונים מבוססת אלבומים

2. רקע:

2.1 תיאור ורקע כללי:

הרעיון לפרויקט נולד מתוך צורך אישי במערכת מסודרת לניהול מתכונים בצורה מאורגנת וברורה, עם אפשרות שליטה מלאה בהרשאות ובתוכן.

המערכת מיועדת לניהול, צפייה ושיתוף מתכונים בצורה מסודרת, מאובטחת ומבוססת הרשאות. המערכת מאפשרת צפייה חופשית לאורחים, יצירה וניהול מתכונים למשתמשים רשומים, וניהול מערכת מלא למנהל.

מטרות המערכת:

- ניהול מתכונים לפי אלבומים
- הפרדת הרשאות (אורח / משתמש / מנהל)
- שמירה מאובטחת של מידע
- יצירת PDF למתכונים
- אפשרות לשדרוג עתידי באמצעות בינה מלאכותית

3. סקירת מצב קיים

מערכות מתכונים קיימות מאפשרות צפייה בלבד או יצירת מתכונים ללא ניהול הרשאות מתקדם. ברוב המערכות אין:

- ניהול אלבומים דינמי
- הפרדת הרשאות מתקדמת
- ממשק ניהול משתמשים
- מערכת סטטיסטיקות

4. מה הפרויקט אמור לחדש או לשפר?

- מערכת הרשאות מבוססת תפקידים
- שיוך מתכון למשתמש יוצר
- ניהול אלבומים ע"י מנהל בלבד

- הפקת PDF אוטומטית
- סטטיסטיקות מערכת

אפשרות לשדרוג AI:

- הצעת כותרת חכמה
- שיפור ניסוח מתכון
- ניתוח פופולריות

5. דרישות מערכת ופונקציונאליות

5.1 דרישות מערכת, סביבת הטמעה ושימוש:

- מערכת Web מבוססת דפדפן
- צד לקוח: React
- צד שרת: .NET Core Web API
- מסד נתונים רלציוני (PostgreSQL)
- סביבת פיתוח מקומית

5.2 שרידות / ביצועים / התמודדות עם עומסים:

- הפרדת שכבות ארכיטקטונית
- שימוש ב-Entity Framework Core
- שימוש בטראנזקציות בעת שמירה
- אינדוקס שדות חיפוש
- תמיכה במספר משתמשים במקביל

5.3 דרישות פונקציונאליות

5.3.1 מנהל המערכת:

- הוספה / עריכה / מחיקת אלבומים
- ניהול כל המתכונים
- ניהול משתמשים (חסימה / ביטול חסימה)
- צפייה בסטטיסטיקות
- גישה למסך ניהול ייעודי

5.3.2 לקוח:

אורח:

- צפייה באלבומים
- צפייה במתכונים
- חיפוש ומיון
- ניסיון הורדה מחייב התחברות

משתמש רשום:

- הוספת מתכון
- בחירה בין קישור חיצוני או טקסט מלא
- העלאת תמונה
- עריכת מתכונים בלבד
- מחיקת מתכונים בלבד
- הורדת מתכון כ-PDF

6. בעיות צפויות במהלך הפיתוח ופתרונות

6.1 תיאור הבעיות:

- בעיה 1: ניהול הרשאות ואבטחה
- בעיה 2: יצירת PDF דינמי

6.2 פתרונות אפשריים:

לבעיה 1:

- פתרון 1: בדיקות בצד לקוח
- פתרון 2: בדיקות בצד שרת + JWT

לבעיה 2:

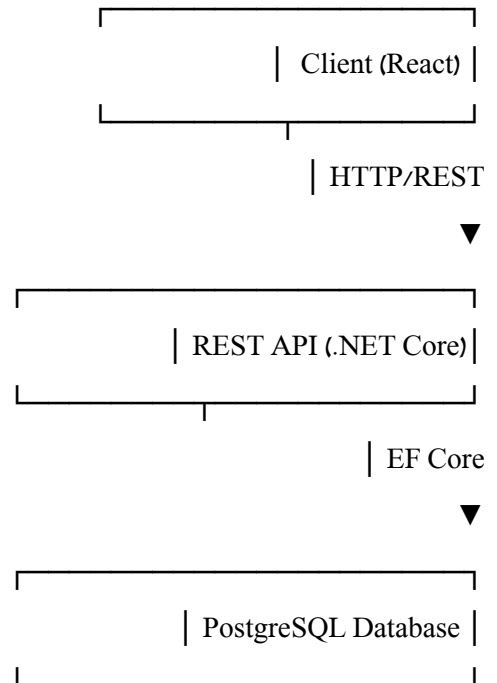
- פתרון 1: יצירת קובץ סטטי
- פתרון 2: יצירה דינמית בצד שרת

6.3 הפתרונות הנבחרים:

- לבעיה 1: JWT + בדיקת Role בצד שרת
- לבעיה 2: יצירה דינמית והחזרה כ-Stream

7. פתרון טכנולוגי נבחר:

7.1 טופולוגית הפתרון



7.2 טכנולוגיות בשימוש

- NET Core.
- Entity Framework Core
- PostgreSQL
- React
- JWT

7.3 שפות הפיתוח

שפות הפיתוח בצד השרת: C#, SQL

שפות הפיתוח בצד הלקוח: JavaScript, HTML, CSS

7.4 ארכיטקטורה נבחרת

: Layered Architecture

Controller → Service → Repository → Database

7.5 חלוקה לתכניות ומודולים

(Separation of Concerns והפרדת אחריות) Clean Architecture המבנה תוכנן בהתאם לעקרונות

בצד השרת:

(, תוך Layered Architecture צד השרת יפותח במבנה מודולרי בהתאם לארכיטקטורת שלוש שכבות) הפרדה ברורה בין שכבות המערכת לצורך תחזוקה, הרחבה ושימוש חוזר בקוד.

המערכת תחולק לשכבות הבאות:

API (Presentation Layer) שכבת

אחראית על:

מהלקוח HTTP קבלת בקשות

(Routing ניהול ניתוב)

JWT אימות משתמשים באמצעות

(Role-Based Authorization בדיקות הרשאה)

(ללקוח Responses החזרת תגובות)

בשכבה זו יוגדרו:

Controllers

DTOs

Middleware

Filters

BLL (Business Logic Layer) שכבת לוגיקה עסקית –

אחראית על:

מימוש חוקי העסק

עסקי) Validation בדיקות תקינות (

עיבוד נתונים

, מיון מותאם וסטטיסטיקות PDF ניהול תהליכים כגון יצירת

בשכבה זו יוגדרו:

Services

Interfaces

מנגנוני חישוב ואלגוריתמים

DAL (Data Access Layer) שכבת גישה לנתונים –

אחראית על:

PostgreSQL תקשורת עם מסד הנתונים

שליפה, שמירה, עדכון ומחיקה של נתונים

Transactions ניהול

Entity Framework Core עבודה מול

בשכבה זו יוגדרו:

Repositories

DbContext

Entity Models

מבנה לוגי של הפרויקט בצד השרת:

API שכבת ה- – Controllers
 (BLL) שכבת הלוגיקה העסקית – Services
 (DAL) שכבת הגישה לנתונים – Data
 Entity-מחלקות ה- – Models
 אובייקטים להעברת נתונים – DTOs
 ניהול גרסאות מסד הנתונים – Migrations

React: בצד הלקוח)

הלקוח יחולק למודולים פונקציונאליים:

(Auth מודול אימות והרשאות)

(Recipes מודול מתכונים)

(Albums מודול אלבומים)

(Admin מודול ניהול)

(Shared Components קומפוננטות משותפות)

(API (Axios Services שירותי

(State (Context / Hooks ניהול

7.6 סביבת השרת

פיתוח מקומי עם אפשרות עתידית להעלאה לשרת ענן.

7.7 ממשק המשתמש/לקוח – GUI

המערכת תכלול:

- דף ראשי עם כרטיסי אלבומים
- עמוד אלבום עם חיפוש ומיון
- עמוד מתכון מפורט
- טופס הוספת מתכון
- מסך ניהול עם טאבים

7.8 ממשקים למערכות אחרות / API

- REST API פנימי
- תמיכה עתידית בשילוב API חיצוני של בינה מלאכותית

7.9 שימוש בחבילות תוכנה

- Entity Framework Core
- JWT Authentication (Microsoft.AspNetCore.Authentication.JwtBearer)
- ספריית PDF (iTextSharp / QuestPDF)
- React Router
- Axios

8. שימוש במבני נתונים וארגון קבצים

8.1 מבני הנתונים

טבלאות מסד הנתונים:

Users	Albums	Recipes
,Id, Username, Email ,PasswordHash, Role, IsBlocked CreatedAt	,Id, Name, Description ImagePath, CreatedAt	Id Name Description RecipeType (Link / Text) Link (nullable) Ingredients (nullable) Instructions (nullable) ImagePath AlbumId (FK) UserId (FK) CreatedAt

קשרים:

- Album → Recipes (אחד לרבים)
- User → Recipes (אחד לרבים)

8.2 שיטת האחסון

- נתונים נשמרים במסד נתונים רלציוני (PostgreSQL)
- תמונות נשמרות בתיקייה בשרת
- נתיב התמונה נשמר במסד הנתונים

8.3 מנגנוני התאוששות מנפילה / קריסה / תמיכה בטראנזקציות

- שימוש ב-Transactions לשמירת אטומיות
- טיפול בחריגות (Exception Handling) בכל שכבה
- מנגנון לוגים (Logging) לתיעוד שגיאות
- Validation בצד שרת למניעת נתונים שגויים

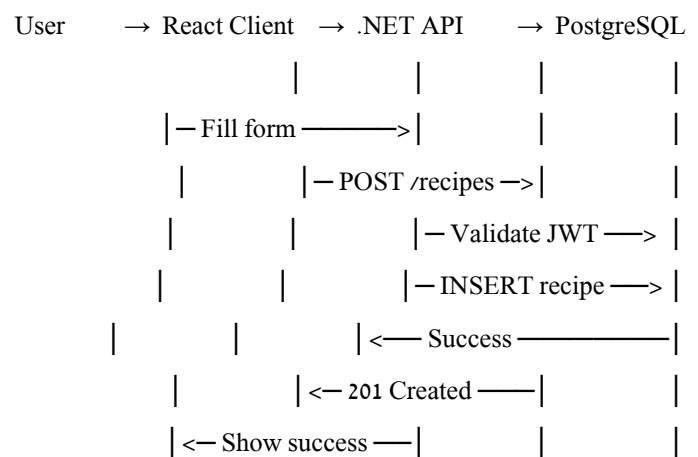
9. תרשימי מערכת מרכזיים

Use Case 9.1

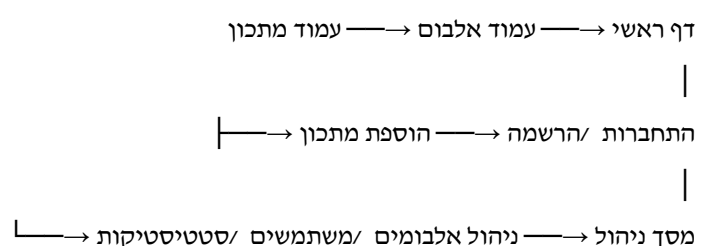
Actor	Use Cases
Guest (אורח)	צפייה באלבומים, צפייה במתכונים, חיפוש ומיון
User (משתמש רשום)	כל פעולות האורח + הוספת מתכון, עריכת מתכון, מחיקת מתכון, הורדת PDF
Admin (מנהל)	כל פעולות המשתמש + ניהול אלבומים, ניהול משתמשים, צפייה בסטטיסטיקות, עריכה/מחיקה של כל מתכון

Sequence Diagram 9.2

תרשים רצף – הוספת מתכון :



9.3 זרימת מסכים



10. תיאור המרכיב האלגוריתמי – חישובי

10.1 איזה בעיה בא לפתור, איך יפתור?

הבעיה: הצגת מתכוני המשתמש בראש הרשימה תוך שמירה על מיון תקין.
 הפתרון: מיון מותאם – תחילה מתכוני המשתמש המחובר, לאחר מכן לפי תאריך יצירה (מהחדש לישן).
 הרחבה עתידית אפשרית: הוספת מנגנון סימון מתכונים כמועדפים ושמירתם לאזור אישי של המשתמש.

10.2 איסוף מידע וניתוחים סטטיסטיים

- מספר מתכונים לכל אלבום
- מספר מתכונים לכל משתמש
- האלבומים הפופולריים ביותר (לפי כמות מתכונים)
- מספר משתמשים רשומים לעומת חסומים
- מספר הורדות PDF

11. תיאור/התייחסות לנושאי אבטחת מידע

- JWT Authentication – אימות משתמשים באמצעות טוקן
- הצפנת סיסמאות (Hash + Salt) – באמצעות BCrypt
- Role-based Authorization – הרשאות לפי תפקיד (Admin / User)
- חסימת משתמשים – מנהל יכול לחסום/לבטל חסימה
- בדיקת בעלות על מתכון בצד שרת – רק הבעלים יכול לערוך/למחוק
- Validation – בדיקת קלט בצד שרת ובצד לקוח

משאבים הנדרשים לפרויקט:

מספר שעות המוקדש לפרויקט: 720

חלוקת עבודה בין חברי הצוות:

הפרויקט מבוצע על ידי סטודנטית יחידה – רבקי גבאי. כל שלבי הפיתוח מבוצעים באופן עצמאי.

ציוד נדרש:

- מחשב נייד / נייד עם מערכת הפעלה Windows 10+
- חיבור אינטרנט

תוכנות נדרשות:

- Visual Studio 2022 / VS Code
- PostgreSQL + pgAdmin
- Node.js + npm
- Git
- Postman (לבדיקת API)

ידע חדש שנדרש ללמוד:

- Entity Framework Core – עבודה עם ORM
- JWT Authentication – מנגנון אימות
- React Hooks & Router – ניהול מצב וניווט
- יצירת PDF דינמי בצד שרת

ספרות ומקורות מידע:

- Microsoft Docs – ASP.NET Core
- React Official Documentation
- PostgreSQL Documentation
- Stack Overflow
- YouTube Tutorials

12. תכנית עבודה ושלבנים למימוש הפרויקט:

החל מתאריך	עד תאריך	שלב
01.02	5.02	יזום
5.02	10.02	ניתוח מבני נתונים
11.02	13.02	איפיון UI
14.02	25.02	כתיבת לוגיקה עסקית
25.02	03.03	עיצוב
04.03	10.03	QA
11.03	15.03	התקנה הטמעה והדרכה

13. תכנון הבדיקות שיבוצעו

בדיקות תהליכיות ברמת משתמש בהן נדרשת המערכת לעמוד (Full Flow)

מספר בדיקה	מס' דרישה במסמך אפיון	מקרי הבדיקה	ידנית/אוטומטית	חשיבות	הערות
1	5.3.2	הרשמה והתחברות – בדיקת תקינות טופס, JWT, ניתוב	ידנית	גבוהה	
2	5.3.2	הוספת מתכון – מילוי טופס, העלאת תמונה, שמירה	ידנית	גבוהה	
3	5.3.1	ניסיון עריכה ללא הרשאה – משתמש מנסה לערוך מתכון של אחר	ידנית	גבוהה	
4	5.3.2	הורדת PDF – לחיצה על כפתור הורדה וקבלת קובץ תקין	ידנית	בינונית	

בדיקות יחידה (unit test)

בדיקות יחידה ומתודולוגיית בדיקות יוגדרו ויישמו בשלב המימוש.

14. בקרת גרסאות (version control)

- Git – ניהול גרסאות מקומי
- GitHub – ניהול גרסאות מרוחק
- ניהול גרסאות בענפים (branches) – ענף נפרד לכל פיצ'ר

הערות הגורם המקצועי מטעם מה"ט:

אישור הגורם המקצועי מטעם מה"ט:

שם: _____ חתימה: _____ תאריך: _____

חתימת המנחה

חתימת הסטודנט

חתימת הסטודנט

_____ רבקי גבאי _____

רבקי גבאי

הערות ראש המגמה במכללה:

אישור ראש המגמה:

שם: _____ חתימה: _____ תאריך: _____

תודות

בראש ובראשונה תודה לבורא עולם שהביאני עד הלום.

תודה מיוחדת למנחה הגב' שרה אבער. על הכוונה, הסבלנות והמשוב לאורך כל שלבי הפיתוח

תודה למשפחה ולחברות. על התמיכה וההבנה בתקופת העבודה על הפרויקט

תודה לכל מי שניסה את המערכת, הגיב והעיר הערות שסייעו לשפר אותה

הקדמה

RecipeBox היא אפליקציית ווב שפותחה כפרויקט גמר.

המערכת נוצרה מתוך הכרת צורך אמיתי: אנשים שאוהבים לבשל מנהלים את המתכונים שלהם בצורה מפוזרת – פתקים, קישורים, הודעות WhatsApp וספרי בישול. אין מקום אחד, מסודר ונגיש, המרכז את כלל המתכונים במקום אחד.

הפתרון המוצע הוא פלטפורמה מרכזית לשמירה, ארגון וגילוי מתכונים, תוך שילוב יכולות בינה מלאכותית המוסיפות ערך למשתמש. בין היכולות: חיפוש מתכונים לפי חומרים, שיפור טקסט, המרת יחידות מידה וכפל כמויות.

הפרויקט מממש ארכיטקטורת Full-Stack מלאה:

- צד לקוח – React + TypeScript

- צד שרת – #ASP.NET Core / C

- מסד נתונים – PostgreSQL

בנוסף, המערכת כוללת מערכת הרשאות תלת-שכבתית (אורח / משתמש / מנהל), ניהול תמונות באמצעות Cloudinary ואינטגרציה עם מודל שפה גדול באמצעות Google AI Studio.

תקציר

פרטים	נושא
קופסת המתכונים – RecipeBox	שם הפרויקט
Full-Stack אפליקציית ווב	סוג המערכת
ניהול מפוזר ולא נגיש של מתכונים בישול	בעיה שנפתרת
(Admin) אורח · משתמש רשום · מנהל	משתמשים
React 19 + TypeScript + Chakra UI	Frontend
ASP.NET Core 9 / C# + Entity Framework	Backend
PostgreSQL (Clever Cloud)	מסד נתונים
Google AI Studio	AI
Cloudinary CDN	תמונות
תוקף 7 ימים, Bcrypt, JWT Bearer	אימות

תוכן עניינים

19	מבוא
20	פירוט הפרויקט – איך המערכת עובדת
22	מבנה נתונים
25	פונקציות ולוגיקה מרכזית
28	מבנה המערכת (Client / Server / DB)
30	מדריך משתמש לפי תפקידים
32	תיאור מסכים
40	סיכום
41	הצעת פרוייקט

1 מבוא

1.1 הרעיון

נולד מתוך בעיה יומיומית: ניהול מתכונים הוא כאב ראש. מתכון שנמצא בפינטרסט, מתכון RecipeBox שסבתא הכתיבה בטלפון, מתכון שנשמר כצילום מסך — כולם פזורים ואי אפשר למצוא אותם ברגע הנכון.

הרעיון היה ליצור מקום אחד שבו שומרים, מארגנים ומגלים מתכונים בקלות. לא רק אחסון, אלא כלי עם שיכול לעזור למצוא מה לבשל לפי מה שיש במקרר, לשפר ניסוח, להמיר יחידות AI ערך מוסף: עוזר מידה ולהכפיל כמויות.

1.2 מה הבעיה שהמערכת פותרת

בעיה	RecipeBox - פתרון
מתכונים מפוזרים בין עשרות מקורות	שמירה מרכזית – טקסט מלא או קישור לאתר חיצוני
אין אפשרות לחפש לפי חומרים	מחפש ויוצר מתכון לפי רשימת חומרים AI עוזר
קשה לשנות כמות מתכון	AI-כפל כמויות $1 \times$ עד $10 \times$ עם עדכון הוראות ההכנה ב
יחידות מידה לא ידועות	AI המרה בין כוסות לגרמים ובחזרה דרך
אין ארגון נושאי	('קינוחים, מנות עיקריות וכו, עוגות) אלבומים מאורגנים
אין מנגנון מועדפים	סימון מועדפים אישי נפרד לכל משתמש

1.3 קהל היעד

המערכת פונה לכל מי שמבשל בבית ורוצה סדר. היא לא מיועדת לטבחים מקצועיים אלא לקהל הרחב בצורה פשוטה AI-אנשים שרוצים לשמור מתכונים משפחה, לגלות רעיונות חדשים ולהשתמש ב — ומועילה.

2 פירוט הפרויקט – איך המערכת עובדת

2.1 סקירה כללית

RecipeBox פותחה במבנה של Single Page Application (SPA), שבו כלל הניווט מתבצע בצד הלקוח ללא טעינה מחדש של דפי המערכת.

צד הלקוח שולח בקשות HTTP אל שרת ה-ASP.NET Core, אשר אחראי על ניהול הלוגיקה העסקית והחזרת נתונים בפורמט JSON.

השרת מתקשר עם מסד הנתונים PostgreSQL וכן עם שירותים חיצוניים, בהם Cloudinary לניהול תמונות ו-Google AI Studio לצורך אינטגרציה עם מודל שפה גדול.

2.2 תהליך הרשמה וכניסה

בעת הכניסה למערכת, המשתמש מגיע לדף הבית ויכול לצפות באלבומים ובמתכונים האחרונים גם ללא ביצוע התחברות.

לחיצה על כפתור "הירשם" פותחת חלון קופץ (LoginModal) מעל הדף הנוכחי, ללא מעבר לעמוד חדש וללא עזיבת המסך הפעיל.

תהליך ההרשמה מתבצע באופן הבא:

- פרטי המשתמש (שם משתמש, כתובת דואר אלקטרוני וסיסמה) נשלחים אל נקודת הקצה POST `/api/auth/register`
- הסיסמה מוצפנת בצד השרת באמצעות BCrypt לפני שמירתה במסד הנתונים
- לאחר הרשמה מוצלחת, השרת מחזיר JWT Token בעל תוקף של שבעה ימים
- ה-Token נשמר ב-localStorage ומצורף באופן אוטומטי לכל בקשת HTTP באמצעות Axios Interceptor
- במקרה של פקיעת תוקף ה-Token, צד הלקוח מנקה את נתוני ההתחברות ומבצע ניתוק אוטומטי של המשתמש

2.3 תהליך הוספת מתכון

משתמש מחובר לוחץ על "+" הוסף מתכון" ומנותב לטופס ייעודי. הטופס תומך בשני מסלולים:

• מתכון טקסט: שם, תיאור, בחירת אלבום, מרכיבים, הוראות הכנה, תמונה (אופציונלי)

• מתכון קישור: שם, תיאור, בחירת אלבום, כתובת URL של האתר החיצוני

בלחיצה על "צור מתכון":

אם הועלתה תמונה – היא נשלחת ל-Cloudinary ומוחזרת כתובת URL.

לאחר מכן כל הנתונים נשמרים ב-PostgreSQL.

בסיום מוצגת אנימציה קונפטי (canvas-confetti), ולאחר 2.5 שניות המשתמש מנותב לאלבום.

2.4 AI תהליך חיפוש עם

המשתמש נכנס לדף "מה יש במקרר?" ומזין רשימת חומרים או שם מתכון.

המערכת שולחת בקשה ל-`api/ai/chat/`.

השרת (AIController) מבצע:

- שלב 1: הלקוח (`geminiService.ts`) שולף את כל המתכונים מה API-ובונה את ה-Prompt-
 - שלב 2: שולח את ה Prompt ל-Google AI Studio-דרך AIController שמשמש כפרוקסי בלבד
 - שלב 3: ה-AI מחפש התאמה קיימת – אם מצא, מחזיר אותה; אם לא, מייצר מתכון חדש
 - שלב 4: המשתמש יכול לצפות במתכון שנמצא, או לשמור את המתכון שנוצר לאלבום שיבחר
- כל קריאות ה-AI עוברות דרך השרת בלבד – מפתח ה-API לא נחשף לדפדפן.

2.5 (Admin) תהליך ניהול

מנהל מתחבר עם חשבון (Admin) ורואה תפריט ניהול נוסף ב-(Navbar).

מדף הניהול הוא יכול:

- צפייה בסטטיסטיקות בזמן אמת:
 - מספר משתמשים
 - מספר מתכונים
 - מספר אלבומים
 - משתמשים חסומים

• יצירה, עריכה של אלבומים

• עריכה ומחיקה של כל מתכון במערכת (לא רק שלו)

- חסימה וביטול חסימה של משתמשים
- משתמש חסום מקבל שגיאה 401 בניסיון התחברות

3 מבנה נתונים

3.1 סקירת הטבלאות

כל הקשרים. Users, Albums, Recipes ו-UserFavorites: מסד הנתונים מכיל ארבע טבלאות מחיקת רשומת הורה מוחקת אוטומטית את כל הצאצאים — ON DELETE CASCADE מוגדרים עם

3.2 Users טבלת

שדה	סוג	תיאור
Id	Guid (PK)	מזהה ייחודי אוטומטי
Email	string, Unique	lowercase-כתובת אימייל — נשמרת ב
UserName	string (nullable)	שם תצוגה אופציונלי
PasswordHash	string	BCrypt סיסמה מוצפנת עם
Role	string	'Admin' או 'User': תפקיד
IsBlocked	bool	false: (ברירת מחדל) האם המשתמש חסום
CreatedAt	DateTime	תאריך ושעת יצירת החשבון

3.3 Albums טבלת

שדה	סוג	תיאור
Id	int (PK, Auto)	מזהה ייחודי עולה אוטומטית
Name	string (max 100)	שם האלבום
Description	string (max 500)	תיאור קצר של האלבום
ImagePath	string (nullable)	Cloudinary-תמונת רקע ב URL
CreatedAt	DateTime	תאריך יצירה

3.4 Recipes טבלת

שדה	סוג	תיאור
Id	int (PK, Auto)	מזהה ייחודי
Name	string (max 200)	שם המתכון
Description	string (max 1000)	תיאור קצר
ImagePath	string (nullable)	Cloudinary-תמונה ב URL
RecipeType	Enum: Link/Text	סוג המתכון — טקסט מלא או קישור חיצוני
Link	string (nullable)	Link אם סוג הוא URL
Ingredients	string (nullable)	Text מרכיבים אם סוג הוא

שדה	סוג	תיאור
Instructions	string (nullable)	Text הוראות הכנה אם סוג הוא
AlbumId	int (FK → Albums)	האלבום שאליו שייך המתכון
UserId	Guid (FK → Users)	המשתמש שיצר את המתכון
CreatedAt	DateTime	תאריך יצירה

3.5 UserFavorites טבלת

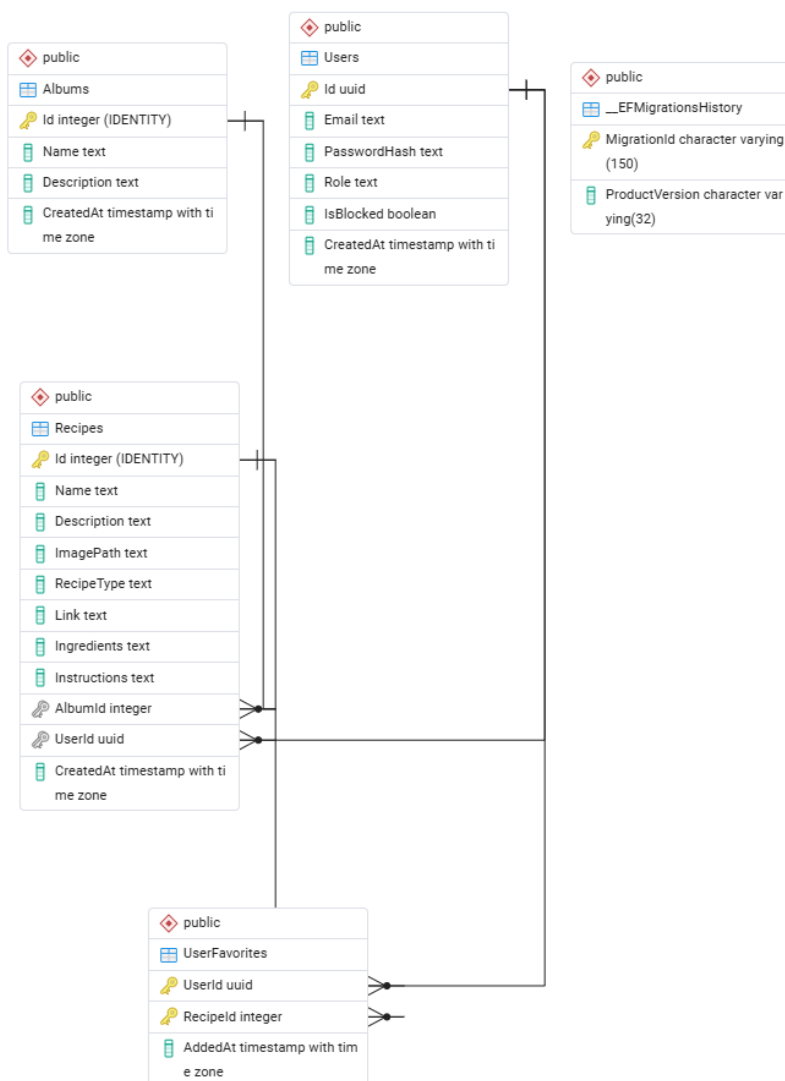
שדה	סוג	תיאור
UserId	Guid (FK → Users, PK חלקי)	מזהה המשתמש
RecipeId	int (FK → Recipes, PK חלקי)	מזהה המתכון
AddedAt	DateTime	מתי המתכון הוסף למועדפים

3.6 קשרים בין הטבלאות

קשר	סוג	הגדרה
User → Recipes	1:N	משתמש יוצר מתכונים רבים. מחיקת משתמש → מחיקת מתכונים
User ↔ Recipe UserFavorites	N:N	משתמש יכול לסמן מועדפים רבים; מתכון יכול להיות מועדף אצל משתמשים רבים

3.7 (EF Core) מיגרציות

- (מרץ 2026) — יצירת הסכמה הראשונית InitialCreate
- עדכון שדות ואילוצים — UpdateModel
- Users לטבלת UserName הוספת שדה — AddUserName



4 פונקציות ולוגיקה מרכזית

4.1 (Auth) אימות והרשאות

המערכת מבוססת על JWT Bearer Authentication. בכל בקשה מוגנת, הלקוח שולח כותרת `<Authorization: Bearer <token>`.

השרת מאמת את ה-Token, מחלץ את ה-UserId ואת ה-Role, ומחליט אם לאפשר את הפעולה.

- הרשמה: Bcrypt — `POST /api/auth/register` מצפין את הסיסמה לפני שמירה
- כניסה: `POST /api/auth/login` — אם הסיסמה תקינה, מוחזר JWT בתוקף 7 ימים
- פקיעת Token: Axios Interceptor מנקה את ה-localStorage ומנתק את המשתמש
- חשבון חסום: השרת מחזיר 401 עם ההודעה "Your account has been blocked"

4.2 (CRUD) ניהול מתכונים

כל פעולה על מתכון עוברת בדיקת בעלות.

רק הבעלים של המתכון או מנהל (Admin) יכולים לערוך או למחוק אותו.

ניסיון של משתמש אחר לבצע פעולה על מתכון מחזיר שגיאה 403 Forbidden.

הרשאה	Endpoint	פעולה
ציבורי	<code>GET /api/recipes</code>	קבלת כל המתכונים
ציבורי	<code>GET /api/recipes/:id</code>	קבלת מתכון בודד
ציבורי	<code>GET /api/recipes/album/:id</code>	מתכונים לפי אלבום
משתמש מחובר	<code>POST /api/recipes</code>	יצירת מתכון
Admin / בעלים	<code>PUT /api/recipes/:id</code>	עריכת מתכון
Admin / בעלים	<code>DELETE /api/recipes/:id</code>	מחיקת מתכון

4.3 ניהול אלבומים

יצירה ועריכת אלבומים מותרת למנהלים בלבד.

אלבומים מנוהלים במערכת על ידי משתמשי Admin בלבד, כולל הוספה ועדכון פרטים.

המתכונים משויכים לאלבומים בהתאם להגדרות המערכת.

4.4 מועדפים

טבלת UserFavorites מנהלת קשר רבים-לרבים בין משתמשים למתכונים. לחיצה על אייקון לב מוסיפה או מסירה מתכון מהמועדפים. עדכון ה-State בלקוח מתבצע מיידית לאחר קבלת תגובת השרת, ללא טעינת הדף מחדש.

• הוספה: POST /api/users/:id/favorites
 (recipeld מועבר ב Body)

- מוחזר סטטוס 409 אם הרשומה כבר קיימת

• הסרה: DELETE /api/users/:id/favorites/:recipeld

• שליפת מועדפים: GET /api/users/:id/favorites

4.5 העלאת תמונות

תמונות אינן נשמרות בשרת המקומי. העלאת התמונות מתבצעת ישירות ל-Cloudinary CDN באמצעות בקשת POST /api/images/upload.

השרת משתמש ב-Cloudinary SDK ומחזיר ללקוח כתובת URL ציבורית, הנשמרת במסד הנתונים בשדה imagePath.

מגבלות העלאה:

• גודל מקסימלי: MB10 • פורמטים נתמכים: JPEG, PNG, GIF, WebP

4.6 AI-יכולות

כל יכולות ה-AI מנותבות דרך AIController בשרת.

הלקוח שולח בקשה ל-POST /api/ai/chat, והשרת מעביר אותה ל-Google AI Studio עם ה-Prompt המתאים ומחזיר את התגובה.

מפתח ה-API נשמר בסביבת השרת בלבד ואינו נחשף לצד הלקוח.

יכולת	תיאור
חיפוש חכם	מחפש מתכונים קיימים לפי חומרים; אם לא נמצא — יוצר מתכון חדש
שיפור טקסט	משפר ניסוח של מרכיבים והוראות הכנה
הצעת שם ותיאור	מייצר שם ותיאור למתכון לפי רשימת המרכיבים
המרת יחידות	ממיר בין כוסות לגרמים ובחזרה, עם הבנה הקשרית
כפל כמויות	מכפיל כמויות $1 \times$ – $10 \times$ ומתאים גם את הוראות ההכנה

4.7 הקלטת קול

בטופס הוספת מתכון ניתן להקליד או להזין הוראות ומרכיבים בעברית באמצעות Web Speech API המובנה בדפדפן.

לחיצה על כפתור המיקרופון מתחילה הקלטה, והטקסט מתומלל ומתווסף לשדה הרלוונטי בזמן אמת. התמיכה בתכונה זו קיימת בעיקר בדפדפן Chrome.

5 מבנה המערכת

5.1 ארכיטקטורה כללית

המערכת בנויה בשלוש שכבות נפרדות עם הפרדת אחריות ברורה. כל שכבה מתקשרת רק עם השכבה הסמוכה לה.

שכבה	טכנולוגיה	אחריות
Client (לקוח)	React 19 + TypeScript + Chakra UI	ממשק משתמש, ניהול מצב, ניתוב
Server (שרת)	ASP.NET Core 9 / C#	לוגיקה עסקית, אימות, REST API
Database	PostgreSQL + EF Core	שמירת נתונים, קשרים, מיגרציות
Cloudinary	שירות חיצוני	CDN-אחסון ותצוגת תמונות ב
Google AI Studio	שירות חיצוני	AI-מודל שפה גדול לכל יכולות ה

5.2 שכבת הלקוח (Client)

הלקוח פועל כ-SPA. תהליך הבנייה מתבצע באמצעות Vite, והקוד כתוב ב-TypeScript. הניווט בין עמודים מנוהל באמצעות React Router, ללא טעינה מחדש של דפים.

- Chakra UI – ספריית קומפוננטות עיצוב עם תמיכה ב-RTL
- Axios – ניהול קריאות HTTP, כולל JWT Interceptor לניהול
- AuthContext – שימוש ב-Context API לניהול מצב התחברות גלובלי
- ProtectedRoute – קומפוננט ניתוב להגנה על נתיבים הדורשים הרשאה

5.3 שכבת השרת (Server)

השרת חשוף על פורטים 6001 (HTTP) ו-7007 (HTTPS). הוא כולל מספר נקודות קצה (Endpoints) המחולקות ל-Controllers:

- AuthController – הרשמה וכניסה
- RecipesController – פעולות CRUD על מתכונים, כולל חיפוש ומיון
- AlbumsController – פעולות CRUD על אלבומים
- UsersController – סטטיסטיקות וחסימת משתמשים-Admin בלבד
- FavoritesController - ניהול מועדפים
- AIController – פרוקסי לכל קריאות Google AI Studio
- ImagesController – העלאת תמונות ל-Cloudinary

Entity Framework Core 9 מנהל את הגישה למסד הנתונים, כאשר שאילתות LINQ מתורגמות אוטומטית ל-SQL.

Swagger / Swashbuckle מספק תיעוד API אוטומטי בסביבת פיתוח.

5.4 Client — מבנה תיקיות

תיקייה /קובץ	תוכן
src/pages/	(א13 קבצי) כל מסכי האפליקציה
src/components/	Navbar, RecipeCard, FavoriteButton...:קומפוננטות משותפות
src/context/	AuthContext — מצב הכניסה הגלובלי
src/services/	geminiService.ts — AI לוגיקת קריאות
src/routes/	ProtectedRoute — הגנת נתיבים

5.5 Server — מבנה תיקיות

תיקייה /קובץ	תוכן
Controllers/	עם כל נקודות הקצה 6 Controllers
Models/	User, Album, Recipe, UserFavorite:מחלקות המודל
Data/	RecipeBoxDbContext—EF הגדרת הטבלאות והקשרים ל Core
Services/	CloudinaryService (AuthService-מוטמעת ב JWT לוגיקת)
appsettings.json	DB, Cloudinary, Google AI Studio, JWT — הגדרות חיבור

6 מדריך משתמש

6.1 משתמש אנונימי - אורח

מבקר שלא נרשם יכול לגלוש באתר ולצפות בתוכן, אך אינו יכול לבצע פעולות שמחייבות זיהוי.

מה הוא רואה:

- דף הבית עם כל האלבומים ו-4 המתכונים האחרונים
- דף כל אלבום עם רשימת המתכונים שבו
- דף מתכון מלא — שם, תיאור, תמונה, מרכיבים, הוראות
- לחיפוש מתכונים AI-דף עוזר ה

מה הוא יכול לעשות:

- לחפש ולסנן מתכונים
- לחיפוש AI-להשתמש בעוזר ה
- Link לפתוח קישורים חיצוניים למתכונים

מה הוא לא יכול לעשות:

- להוסיף, לערוך או למחוק מתכונים
- לסמן מועדפים
- של מתכון PDF להוריד
- AI-לשמור מתכון שנוצר ב

6.2 משתמש רשום (User)

מציג כפתור Navbar-לאחר הרשמה וכניסה, המשתמש מקבל גישה לכל יכולות האפליקציה. ה. מועדפים 'ותפריט משתמש עם אפשרות התנתקות'

מה הוא רואה:

- את כל מה שרואה אורח
- כפתור לב (מועדף) (בכל מתכון
- כפתורי ערוך/מחק על המתכונים שלו בלבד
- בדף מתכון) המרת יחידות, כפל כמויות) AI פס כלי
- דף מועדפים אישי

מה הוא יכול לעשות:

- הוספת מתכון — טקסט מלא עם מרכיבים והוראות, או קישור לאתר חיצוני
- (JPEG/PNG/GIF/WebP — MB עד 10) העלאת תמונה למתכון
- עריכת ומחיקת המתכונים שלו בלבד
- סימון מתכונים כמועדפים והסרתם
- לחיפוש ויצירת מתכונים AI שימוש בעוזר
- AI-שיפור טקסט מרכיבים/הוראות ב
- AI-הצעת שם ותיאור למתכון חדש ב
- המרת יחידות מידה) כוסות ↔ גרמים (בדף מתכון

- כפל כמויות מתכון 1× עד 10×
- הקלטת קול בעברית להזנת מרכיבים/הוראות
- שיתוף קישור למתכון (העתקה ללוח)
- PDF-הדפסת מתכון ל

6.3 (Admin) מנהל

עם 4 פריטים. כל Navbar-הוא מקבל תפריט ניהול ייעודי ב. Role='Admin' מנהל הוא משתמש עם ההגנות נבדקות גם בשרת — לא ניתן לעקוף אותן מהדפדפן

מה הוא רואה:

- את כל מה שרואה משתמש רשום
- כפתורי ערוך/מחק על כל מתכון במערכת
- Admin-ב Navbar תפריט

מה הוא יכול לעשות — מעבר למשתמש רגיל

- יצירת ועריכת אלבומים
- עריכת ומחיקת כל מתכון, לא רק שלו
- צפייה בסטטיסטיקות מערכת בזמן אמת
- חסימת משתמשים חריגים (כפתור 'חסום' בטבלת משתמשים)
- ביטול חסימת משתמשים

6.4 טבלת השוואת הרשאות

Admin	משתמש	אורח	פעולה
✓	✓	✓	צפייה במתכונים ואלבומים
✓	✓	✓	חיפוש וסינון
✓	✓	✓	חיפוש – AI עוזר
✓	✓	—	הוספת מתכון
כולם	שלו בלבד	—	עריכת מתכון
כולם	שלו בלבד	—	מחיקת מתכון
✓	✓	—	מועדפים
✓	✓	✓	בדף מתכון AI יכולות
✓	✓	—	PDF הורדת
✓	—	—	ניהול אלבומים
✓	—	—	ניהול משתמשים
✓	—	—	סטטיסטיקות מערכת

7 תיאור מסכים

האפליקציה כוללת 13 מסכים: 4 ציבוריים, 3 למשתמשים מחוברים, 4 מסכי ניהול, ו-2 רכיבי Layout גלובליים.

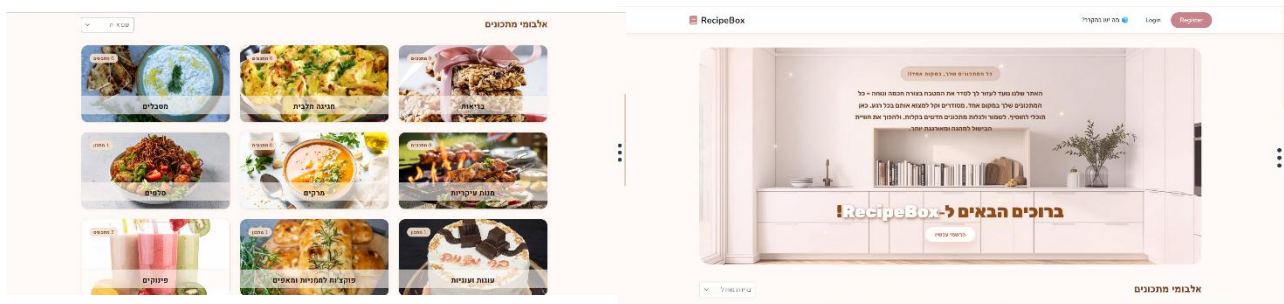
7.1 מפת מסכים

הרשאה	נתיב	מסך
ציבורי	/	דף הבית
ציבורי	/albums/:id	דף אלבום
ציבורי (חלק מהפעולות דורש כניסה)	/recipes/:id	דף מתכון
ציבורי (שמירה דורשת כניסה)	/ai	AI עוזר
ציבורי	גלובלי Modal	כניסה/הרשמה
משתמש מחובר	/recipes/new	הוספת מתכון
Admin / בעלים	/recipes/:id/edit	עריכת מתכון
משתמש מחובר	/favorites	מועדפים
Admin	/admin/stats	ניהול – סטטיסטיקות
Admin	/admin/albums	ניהול – אלבומים
Admin	/admin/recipes	ניהול – מתכונים
Admin	/admin/users	ניהול – משתמשים

7.2 (HomePage) דף הבית

דף הנחיתה הראשי מציג Hero Banner עם קריאה לפעולה, רשת כרטיסיות של אלבומים עם אפשרות מיון (לפי שם או מספר מתכונים), ורשימה של ארבעת המתכונים האחרונים שנוספו.

- משתמש לא מחובר: מוצג כפתור "הירשם עכשיו" ב-Hero Banner
- משתמש מחובר: מוצג כפתור "+ הוסף מתכון"
- לחיצה על אלבום: ניווט ל-/albums/:id
- מיון אלבומים מתבצע בצד הלקוח בלבד, ללא קריאות API נוספות



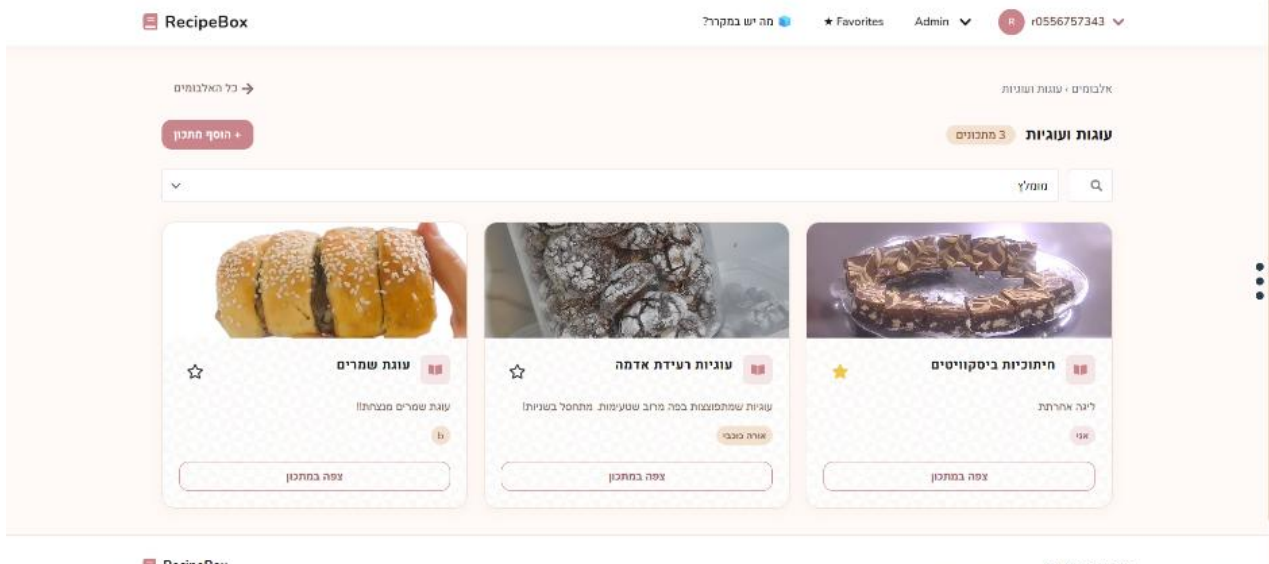
7.3 (AlbumPage) דף אלבום

העמוד מציג את כל המתכונים של האלבום בפריסת רשת.

העמוד כולל שורת חיפוש, אפשרויות מיון ומספר המתכונים הכולל.

כל כרטיס מתכון מציג שם, תמונה וכפתור מועדף.

- חיפוש ומיון מתבצעים בצד הלקוח באמצעות useMemo – ללא קריאות API נוספות
- לחיצה על כפתור מועדף מעדכנת את המצב מיידית ללא רענון דף



7.4 (RecipePage) דף מתכון

המסך המרכזי של האפליקציה מציג את פרטי המתכון המלאים.

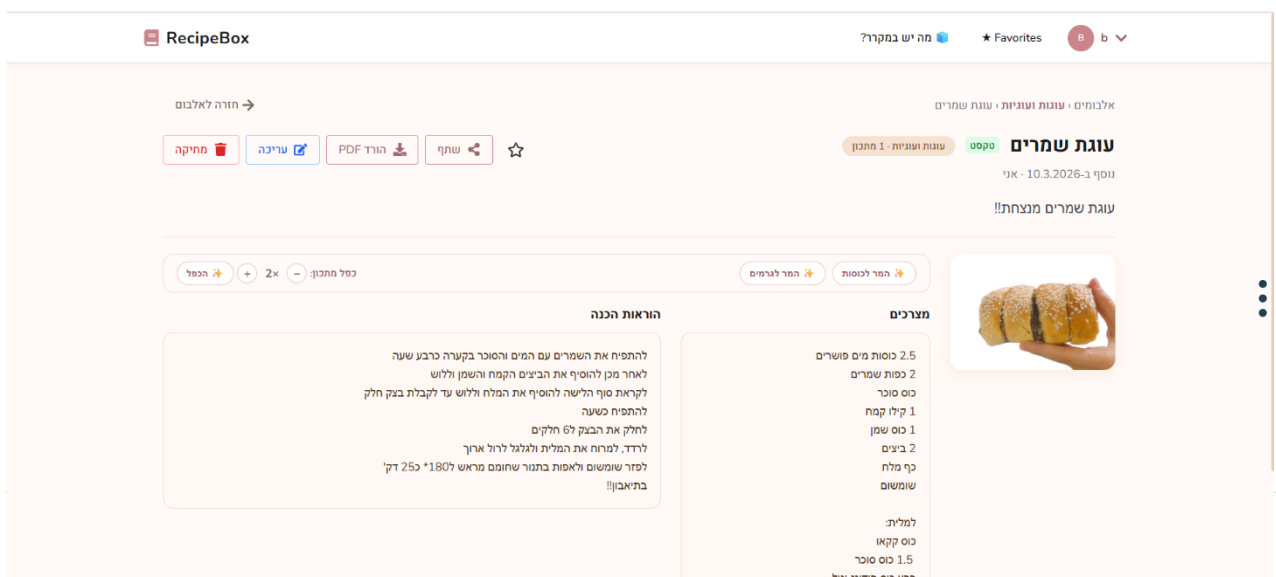
במתכונים טקסט מוצג פס כלי AI. במתכונים קישור מוצג כפתור "פתח מתכון" לאתר החיצוני.

• פס AI (טקסט בלבד): המרת יחידות (כוסות ↔ גרמים), כפל כמויות $1 \times$ – $10 \times$, כפתור איפוס

• כפתור שיתוף: מעתיק את ה-URL ללוח ומחליף לטקסט "הועתק!"

• הורדת PDF: מפעיל window.print() ודורש התחברות

• פעולות עריכה/מחיקה מוצגות רק לבעלים ולמנהל

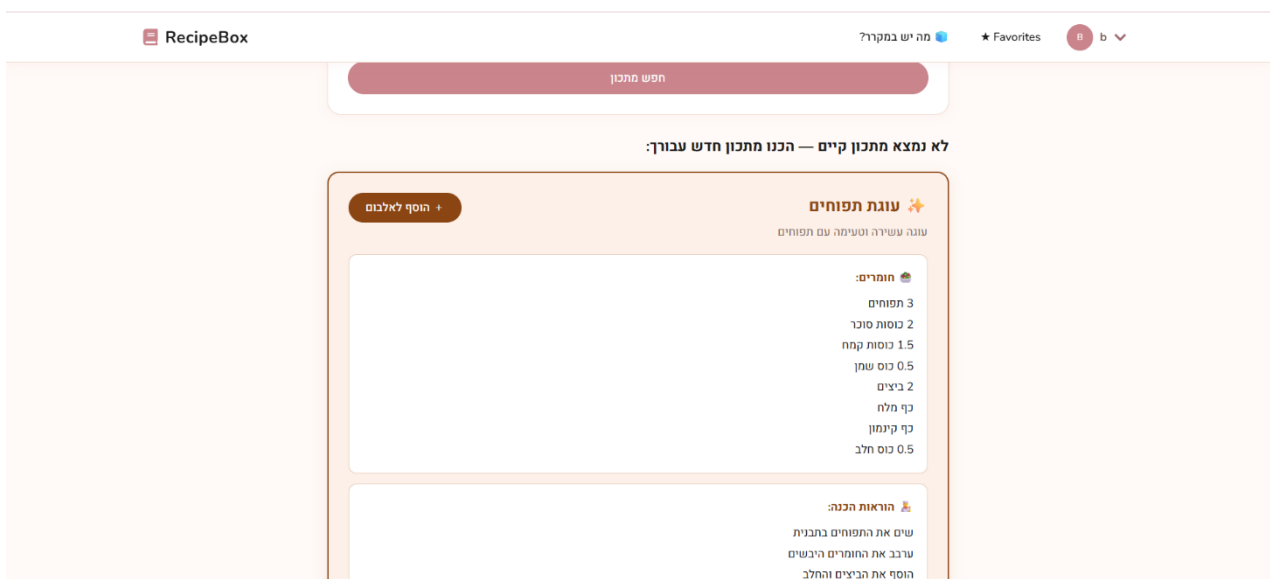
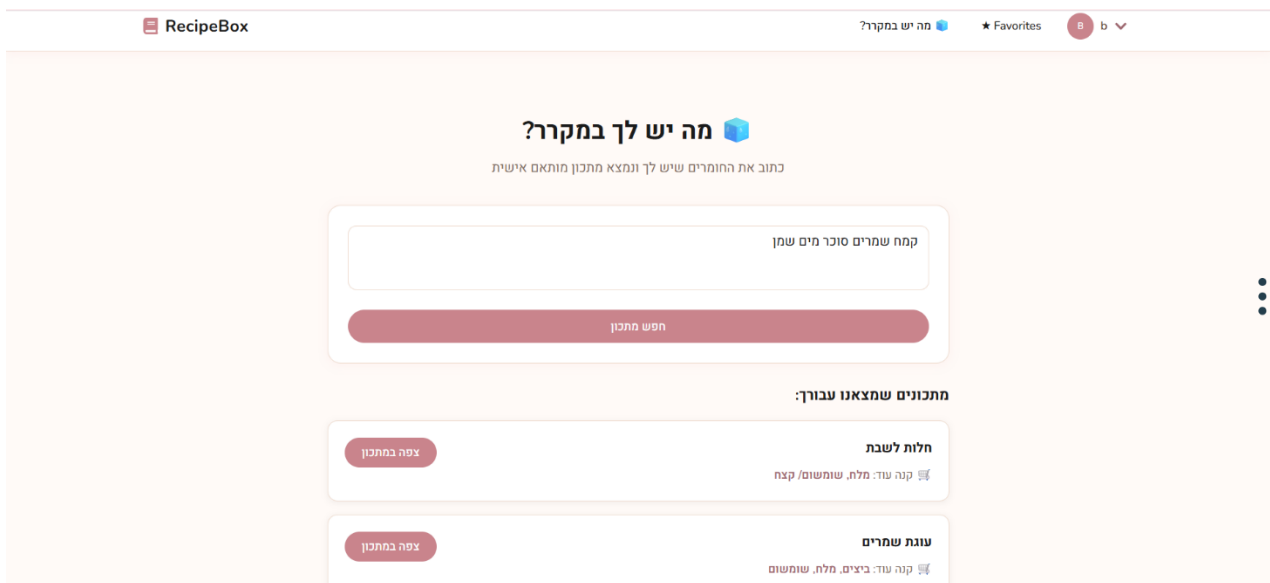


7.5 ('מה יש לך במקרר' – AIPage) עוזר

המשתמש מזין חומרים שיש לו (או שם מתכון).

המערכת מחפשת מתכונים מתאימים בבסיס הנתונים, ואם לא נמצא — נוצרת הצעה למתכון חדש.

- נמצא מתכון קיים: מוצג כפתור "צפה במתכון" עם ניווט ל-`recipes/:id/`
- נוצר מתכון חדש: מוצג הטקסט המלא עם כפתור "+" הוסף לאלבום" (למשתמשים מחוברים)
- משתמש לא מחובר: מוצג כפתור "התחבר לשמירה" שפותח את `LoginModal`



7.6 (AddRecipePage / EditRecipePage) טופס מתכון

טופס יצירה ועריכה משתמש בקומפוננטה משותפת (RecipeForm). בעת עריכה השדות מאוכלסים מראש.

הטופס כולל שלוש יכולות מתקדמות:

- הקלטת קול: כפתור מיקרופון ליד שדות מרכיבים והוראות, עם תמלול בעברית בזמן אמת
- שיפור AI: כפתור "שפר" שולח את התוכן ל-AI ומחזיר גרסה משופרת, עם אפשרות לביטול
- הצעת שם: כפתור "הצע שם ותיאור לפי המרכיבים" שממלא אוטומטית את שני השדות
- בסיום יצירת מתכון: מוצגת אנימצית קונפטי ולאחריה ניווט אוטומטי לאלבום

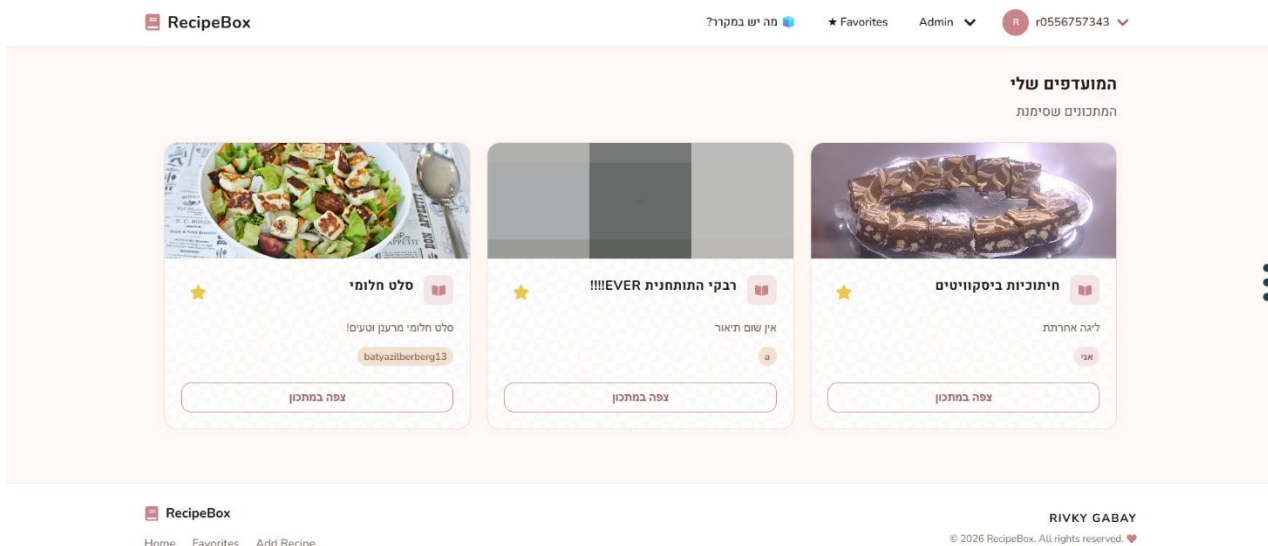
The screenshot shows the 'הוספת מתכון חדש' (Add New Recipe) form in the RecipeBox application. The form is divided into several sections:

- שם המתכון *** (Recipe Name *): A text input field.
- רוטב לחסה** (Lentil Sauce): A text input field.
- סוג מתכון *** (Recipe Type *): A dropdown menu with options like 'קישור' (Link), 'טקסט מלא' (Full Text), and 'שפר' (Improve).
- אלבום** (Album): A section for adding images.
- מרכיבים *** (Ingredients *): A list of ingredients with a 'שפר' (Improve) button.
- הוראות הכנה *** (Instructions *): A text area for instructions.
- הכנס את החומרים** (Add Ingredients): A button to add ingredients.
- הצע שם ותיאור לפי המרכיבים** (Suggest name and description based on ingredients): A button to generate suggestions.
- תמונה (אופציונלית)** (Image (optional)): A section for adding a recipe image.
- URL כתובת** (URL address): A text input field.
- העלאת קובץ** (Upload file): A button to upload a file.

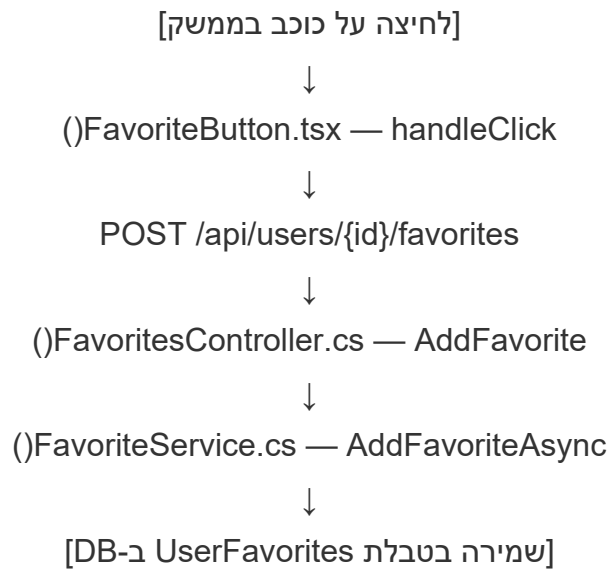
7.7 (FavoritesPage) דף מועדפים

העמוד מציג את כל המתכונים שסומנו כמועדפים על-ידי המשתמש המחובר.

לחיצה על כפתור מועדף מסירה את המתכון מהרשימה באופן מיידי, כאשר ה-State המקומי מתעדכן ללא קריאות API נוספות.



זרימה מלאה — ממשק עד מסד הנתונים



צילומי קוד לפי שכבות

```

1 namespace RecipeBoxServer.Models;
2
3 12 references
4 public class UserFavorite
5 {
6     9 references
7     public Guid UserId { get; set; }
8     6 references
9     public int RecipeId { get; set; }
10    2 references
11    public DateTime AddedAt { get; set; } = DateTime.UtcNow;
12
13    1 reference
14    public User User { get; set; } = null!;
15    17 references
16    public Recipe Recipe { get; set; } = null!;
17 }
  
```

טבלת הקשר בין משתמש למתכון (User–Recipe Mapping Table). המפתח הראשי (Composite Primary Key) מורכב משני שדות: UserId ו-RecipeId, ובכך מובטח שמשתמש לא יוכל לסמן את אותו המתכון כמועדף יותר מפעם אחת.

```

FavoriteService.cs U, M X
server > RecipeBoxServer > Services > FavoriteService.cs > FavoriteService > AddFavoriteAsync
8 public class FavoriteService : IFavoriteService
    2 references
17 public async Task<bool> AddFavoriteAsync(Guid userId, int recipeId)
18 {
19     var userExists = await _db.Users.AnyAsync(u => u.Id == userId);
20     if (!userExists)
21         throw new InvalidOperationException("User not found.");
22
23     var recipeExists = await _db.Recipes.AnyAsync(r => r.Id == recipeId);
24     if (!recipeExists)
25         throw new InvalidOperationException("Recipe not found.");
26
27     var alreadyFavorited = await _db.UserFavorites
28         .AnyAsync(uf => uf.UserId == userId && uf.RecipeId == recipeId);
29
30     if (alreadyFavorited)
31         return false;
32
33     _db.UserFavorites.Add(new UserFavorite
34     {
35         UserId = userId,
36         RecipeId = recipeId,
37         AddedAt = DateTime.UtcNow
38     });
39
40     await _db.SaveChangesAsync();
41     return true;
42 }
43
    2 references

```

שכבת ה-API (API Layer).

מחזיר 409 Conflict כאשר המתכון כבר קיים ברשימת המועדפים.

הפונקציה CanAccessUser מבצעת בקרת הרשאות ומבטיחה שרק בעל החשבון או מנהל מערכת (Admin) יוכלו לבצע פעולות על המועדפים.

```

FavoritesController.cs U X
server > RecipeBoxServer > Controllers > FavoritesController.cs > ...
13 public class FavoritesController : ControllerBase
31 public async Task<IActionResult> AddFavorite(Guid userId, [FromBody] AddFavoriteDto dto)
32 {
33     if (!CanAccessUser(userId)) return Forbid();
34     try
35     {
36         var added = await _favoriteService.AddFavoriteAsync(userId, dto.RecipeId);
37         if (!added) return Conflict(new { message = "Recipe is already in favorites." });
38         return Ok(new { message = "Added to favorites." });
39     }
40     catch (InvalidOperationException ex) { return BadRequest(new { message = ex.Message }); }
41 }
42
43 [HttpDelete("{recipeId:int}")]
    0 references
44 public async Task<IActionResult> RemoveFavorite(Guid userId, int recipeId)
45 {
46     if (!CanAccessUser(userId)) return Forbid();
47     var removed = await _favoriteService.RemoveFavoriteAsync(userId, recipeId);
48     if (!removed) return NotFound(new { message = "Favorite not found." });
49     return NoContent();
50 }
51
    3 references
52 private bool CanAccessUser(Guid userId)
53 {
54     var currentIdStr = User.FindFirstValue(ClaimTypes.NameIdentifier);
55     if (!Guid.TryParse(currentIdStr, out var currentId)) return false;
56     return currentId == userId || User.IsInRole("Admin");
57 }

```

שכבת הלוגיקה העסקית (Business Logic Layer).

בודקת שהמשתמש והמתכון קיימים, מבצעת בדיקת כפילות מול מסד הנתונים לפני ההוספה, ויוצרת רשומת UserFavorite הכוללת את תאריך ההוספה.

```

FavoriteButton.tsx
client > src > components > recipes > FavoriteButton.tsx > ...
14 export default function FavoriteButton({ recipeId, isFavorite, onToggle }: Props) {
15
16   const handleClick = async () => {
17     if (!isAuthenticated) {
18       openLogin();
19       return;
20     }
21     if (loading) return;
22     setLoading(true);
23     try {
24       if (active) {
25         await favoriteService.removeFavorite(user!.userId, recipeId);
26         setActive(false);
27         onToggle?.(false);
28       } else {
29         await favoriteService.addFavorite(user!.userId, recipeId);
30         setActive(true);
31         onToggle?.(true);
32       }
33     } catch (err: any) {
34       // 409 = already favorited - treat as favorited
35       if (err?.response?.status === 409) {
36         setActive(true);
37         onToggle?.(true);
38       }
39     } finally {
40       setLoading(false);
41     }
42   };
43 }
44
45
46
47

```

רכיב צד לקוח (Client-Side Component).

ממוש בגישת Optimistic UI – הלב משנה צבע מיד עם הלחיצה עוד לפני קבלת תשובת השרת. במקרה של Conflict 409, המערכת מתייחסת לכך כהצלחה מאחר שהמתכון כבר קיים ברשימת המועדפים.

7.8 (Admin) מסכי ניהול

/admin/stats סטטיסטיקות

כרטיסי מידע: סה"כ משתמשים, מתכונים, אלבומים, משתמשים חסומים. רשימת 10 המתכונים שנוצרו 4 לאחרונה עם קישור לכל אחד.

/admin/albums ניהול אלבומים

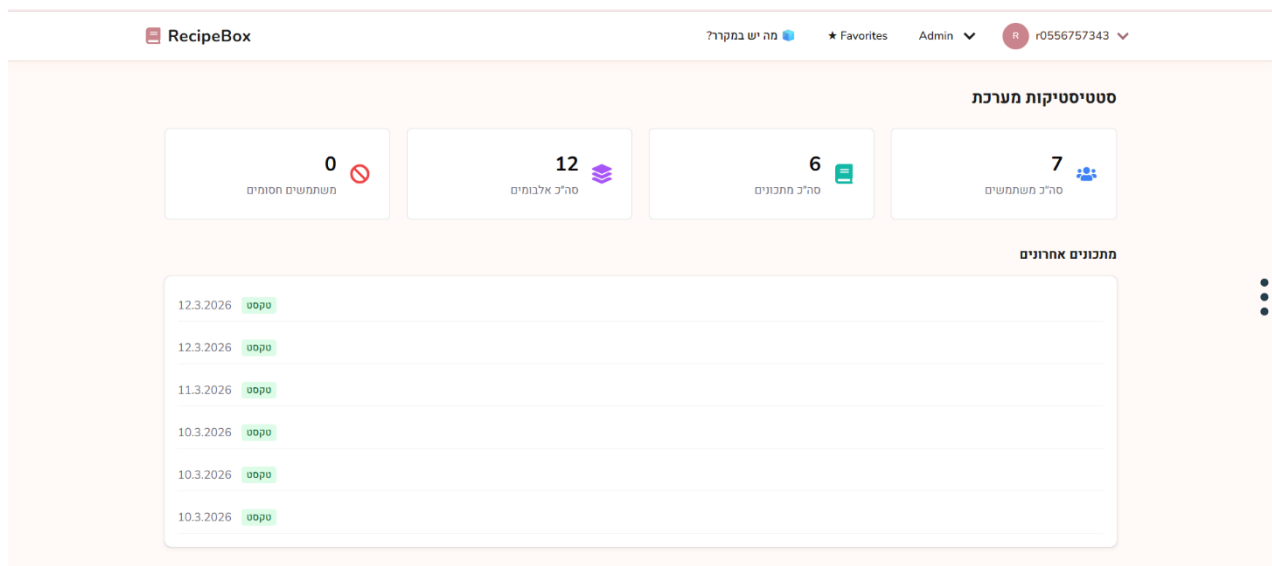
לחיצה על עט (ערוך) — מאכלס את הטופס. inline טבלת כל האלבומים. לחיצה על '+' פותחת טופס

/admin/recipes ניהול מתכונים

טבלת כל המתכונים במערכת עם חיפוש ומיון. כל שורה כוללת כפתורי ערוך (→ /recipes/:id/edit) ומחק.

/admin/users ניהול משתמשים;

טבלת כל המשתמשים: אימייל, תפקיד, מספר מתכונים, סטטוס (ירוק=פעיל, אדום=חסום). (כפתורי חסימה/ביטול מוסתרים עבור מנהלים).



7.9 Navbar ו-LoginModal

ה-Navbar נטען בכל דף כחלק מה-Layout.

הוא מתאים את עצמו לסטטוס ההתחברות ולתפקיד המשתמש.

LoginModal הוא חלון צף גלובלי שניתן לפתוח מכל מקום ב-SPA מבלי לאבד את ההקשר הנוכחי.

הוא תומך בשני מצבים: התחברות והרשמה, עם מעבר בין טאבים.

8 סיכום

8.1 מה למדתי

- פרויקט RecipeBox לימד אותי כיצד לבנות מערכת Full-Stack מלאה מהרעיון ועד לפריסה. בפועל, תהליך הפיתוח הדגים מספר לקחים שלא היו ברורים בתחילה:
- הפרדת אחריות (Separation of Concerns) – כאשר כל שכבה מבצעת בדיוק את תפקידה, הקוד הופך לקל יותר לתחזוקה ולהרחבה
- אבטחה בעומק – הגנה בצד הלקוח (ProtectedRoute, הסתרת כפתורים) אינה מספיקה; השרת חייב לבצע בדיקות הרשאה באופן עצמאי ומלא
- עדכון State מקומי – ביצוע עדכונים מיידיים ל-State (ללא קריאות API נוספות) משפר משמעותית את חוויית המשתמש
- שימוש ב-AI כשירות – שילוב מודל שפה גדול כ-API חיצוני, תוך שמירה על מפתח ה-API בצד השרת בלבד, הוא פתרון פשוט ויעיל

8.2 אתגרים ופתרונות

פתרון שיושם	אתגר
ובדיקת בעלות בשרת [Authorize] + בלקוח ProtectedRoute	ניהול הרשאות בשתי שכבות
מחיקה — FK-בכל קשרי ה CASCADE DELETE אוטומטית ועקבית	עקביות נתונים בעת מחיקה
כפרוקסי — הלקוח לא רואה את המפתח לעולם AIController	Google AI Studio חשיפת מפתח
לניקוי אוטומטי Axios Interceptor + localStorage ב-JWT	Session ב-SPA ניהול
מקומי לאחר קבלת תגובת השרת — ללא קריאה State עדכון נוספת	עדכון מועדפים גורם לרענון
המובנה בדפדפן — תמלול ישיר לשדות Web Speech API המתכון	הוספת הקלטת קול בעברית

8.3 הישגים מרכזיים

- מערכת אימות והרשאות מלאה ומאובטחת עם JWT ו-BCrypt
- אינטגרציה פונקציונלית עם שלושה שירותי ענן: Cloudinary, PostgreSQL (Clever Cloud) ו-Google AI Studio
- חמש יכולות AI ששולבו ישירות בזרימת השימוש
- ממשק ניהול מלא הכולל סטטיסטיקות, ניהול תוכן וניהול משתמשים
- ארכיטקטורה נקייה ומודולרית הניתנת להרחבה בקלות

8.4 המלצות להמשך פיתוח

תיאור	הרחבה
יאפשר חידוש Refresh Token. המשתמש מנותק JWT כיום עם פקיעת שקוף	Refresh Tokens
תהפוך את המערכת לפלטפורמה Rating/Comment הוספת מודל חברתית	דירוגים ותגובות
קלט — תפחית עלויות ותשפר מהירות hash לפי AI שמירת תשובות	Cache AI לתשובות
לחיפוש מדויק PostgreSQL Full-Text Search-פשוט ל LIKE-שדרוג מ יותר	Full-Text Search
לחווית שמירה טובה יותר (window.print לא) PDF-ייצוא אמיתי ל	מוגמר PDF ייצוא
API-לנקודות ה Unit Tests ל-Services + Integration Tests	בדיקות אוטומטיות

8.5 סיכום כללי

RecipeBox היא מערכת שפותחה מהרעיון ועד לפריסה כפרויקט גמר.

המערכת מדגימה שליטה בכלים הנדרשים לפיתוח מערכות Web מודרניות:

React עם ASP.NET Core, TypeScript עם PostgreSQL, C# עם Entity Framework, אימות JWT, הרשאות מבוססות תפקיד, ושילוב שירותי ענן ובינה מלאכותית.

הפרויקט עמד ביעדיו: ממשק אינטואיטיבי, שרת מאובטח עם לוגיקה עסקית ברורה, ויכולות AI המוסיפות ערך אמיתי — כחלק אינטגרלי מהמוצר ולא כתוספת חיצונית בלבד.